

Software Development and Construction

Lab Final



SUBMITTED BY:

Muhammad Saleh Janjua (B22F0086SE017)

SUBMITTED TO:

Ms. Zainab

**DEPARTMENT OF SOFTWARE ENGINEERING
PAK-AUSTRIA INSTITUTE OF APPLIED SCIENCE**

Software Development and Construction

Question 1: Secure Eureka Server with Basic Authentication

Objective

To configure a **Eureka Server** with basic authentication, ensuring only authorized services can register.

Implementation Steps

1. Project Setup

- Created a Spring Boot project via [SpringInitializr](https://start.spring.io) with dependencies:
 - Eureka Server
 - Spring Security

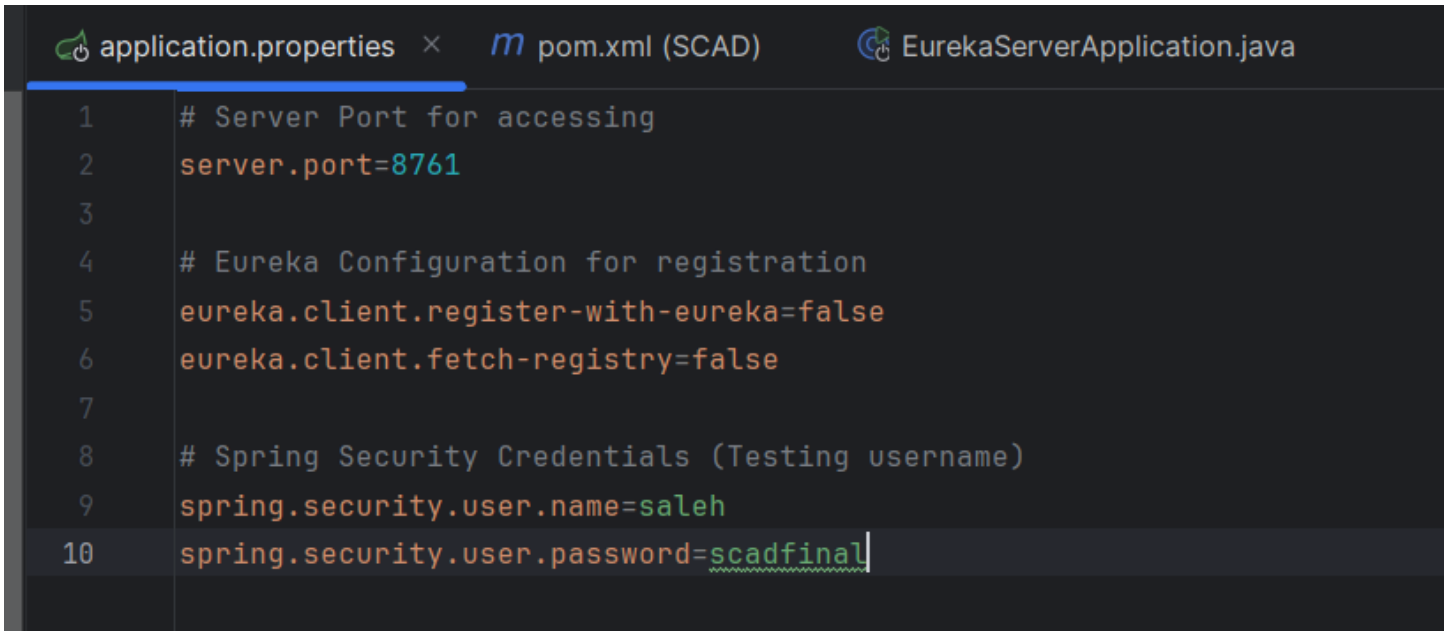
The screenshot shows the Spring Initializr web form at start.spring.io. The form is configured for a Spring Boot project with the following settings:

- Project:** ☐ Gradle - Groovy, ☐ Gradle - Kotlin, ☒ Maven
- Language:** ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot:** ☐ 3.5.0 (SNAPSHOT), ☐ 3.5.0 (RC1), ☐ 3.4.6 (SNAPSHOT), ☒ 3.4.5
- Project Metadata:**
 - Group: com.labfinal
 - Artifact: SCAD
 - Name: SCAD
 - Description: Lab final for Software Construction and development.
 - Package name: com.labfinal.SCAD
 - Packaging: ☒ Jar, ☐ War
 - Java: ☐ 24, ☐ 21, ☒ 17
- Dependencies:**
 - Spring Security** (SECURITY): Highly customizable authentication and access-control framework for Spring applications.
 - Eureka Server** (SPRING CLOUD DISCOVERY): spring-cloud-netflix Eureka Server.

Buttons at the bottom: **GENERATE** (CTRL + G), **EXPLORE** (CTRL + SPACE), and a menu button (...).

Software Development and Construction

2. Configuration

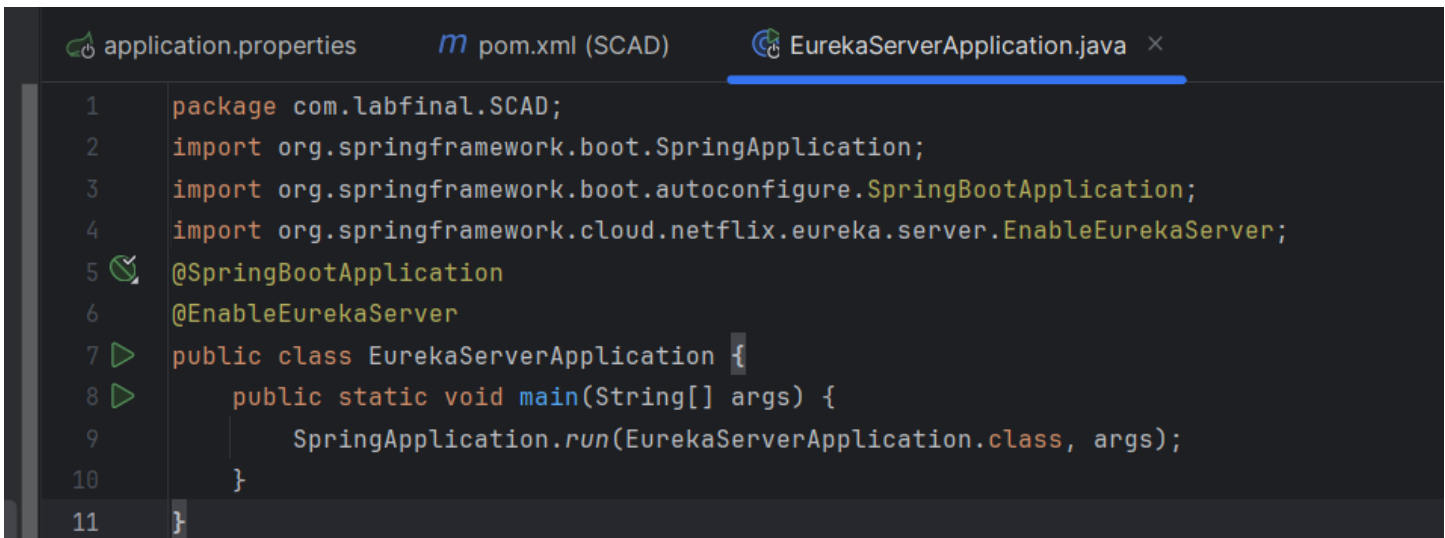


The screenshot shows an IDE with three tabs: 'application.properties', 'pom.xml (SCAD)', and 'EurekaServerApplication.java'. The 'application.properties' tab is active and displays the following configuration:

```
1 # Server Port for accessing
2 server.port=8761
3
4 # Eureka Configuration for registration
5 eureka.client.register-with-eureka=false
6 eureka.client.fetch-registry=false
7
8 # Spring Security Credentials (Testing username)
9 spring.security.user.name=saleh
10 spring.security.user.password=scadfinal
```

3. Code Implementation

- Added `@EnableEurekaServer` annotation to the main class:



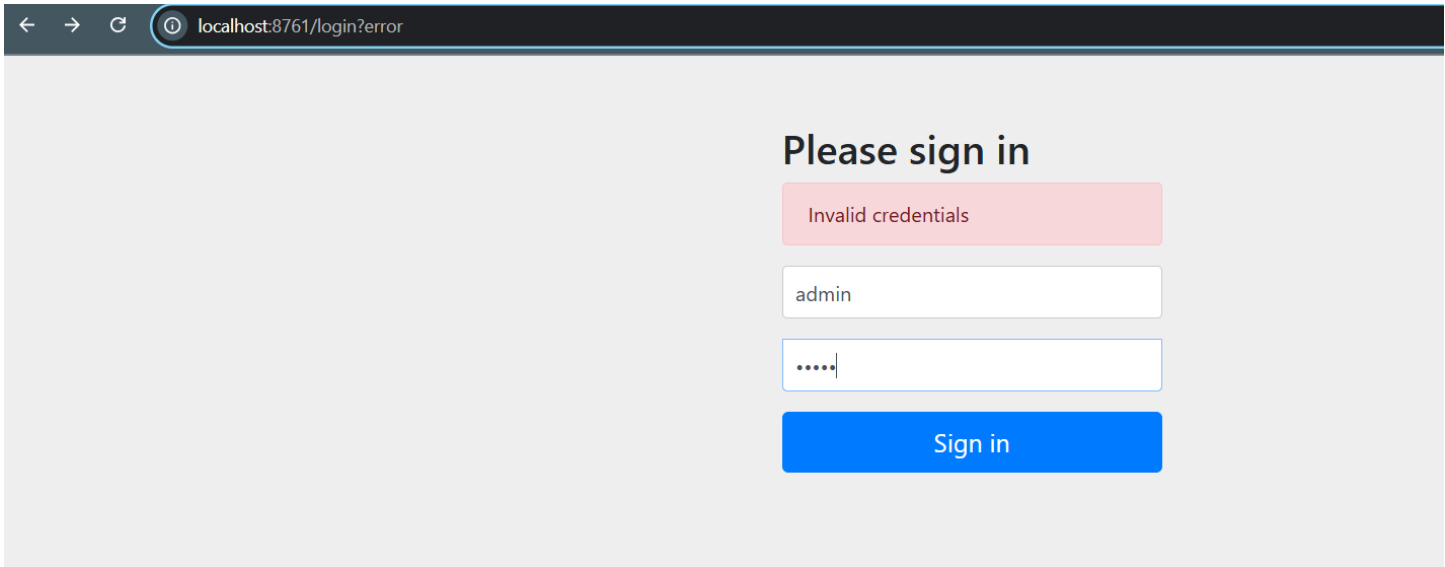
The screenshot shows an IDE with three tabs: 'application.properties', 'pom.xml (SCAD)', and 'EurekaServerApplication.java'. The 'EurekaServerApplication.java' tab is active and displays the following code:

```
1 package com.labfinal.SCAD;
2 import org.springframework.boot.SpringApplication;
3 import org.springframework.boot.autoconfigure.SpringBootApplication;
4 import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
5 @SpringBootApplication
6 @EnableEurekaServer
7 public class EurekaServerApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(EurekaServerApplication.class, args);
10    }
11 }
```

Software Development and Construction

4. Testing

- Ran the server and accessed <http://localhost:8761>.



← → ↻ localhost:8761/login?error

Please sign in

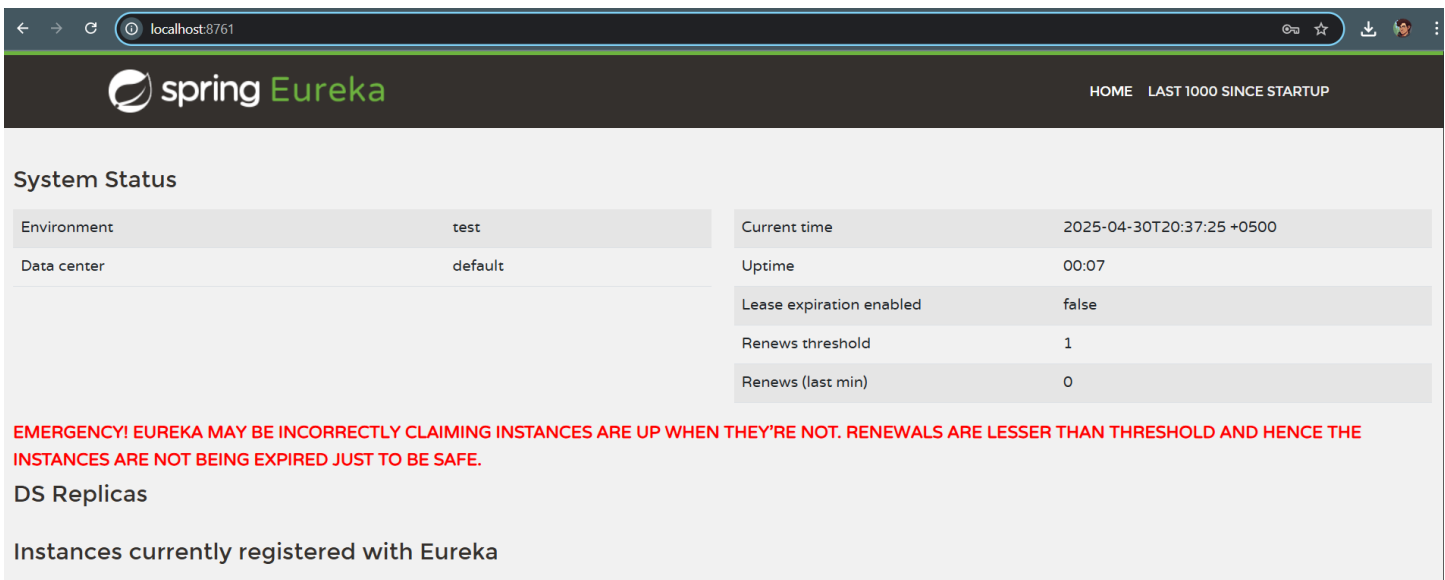
Invalid credentials

admin

.....

Sign in

- Logged in with credentials saleh/scadfinal.
- Directs us to the Eureka Dashboard.



← → ↻ localhost:8761

spring Eureka

HOME LAST 1000 SINCE STARTUP

System Status

Environment	test
Data center	default

Current time	2025-04-30T20:37:25 +0500
Uptime	00:07
Lease expiration enabled	false
Renews threshold	1
Renews (last min)	0

EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.

DS Replicas

Instances currently registered with Eureka

Software Development and Construction

Question 2: JPA Application Registered with Eureka

Objective

To develop a **Spring Boot JPA application** that auto-registers with the secured Eureka Server and performs CRUD operations.

Implementation Steps

1. Project Setup

- Created a Spring Boot project with dependencies:
 - Eureka Client
 - Spring Web
 - Spring Data JPA
 - H2 Database

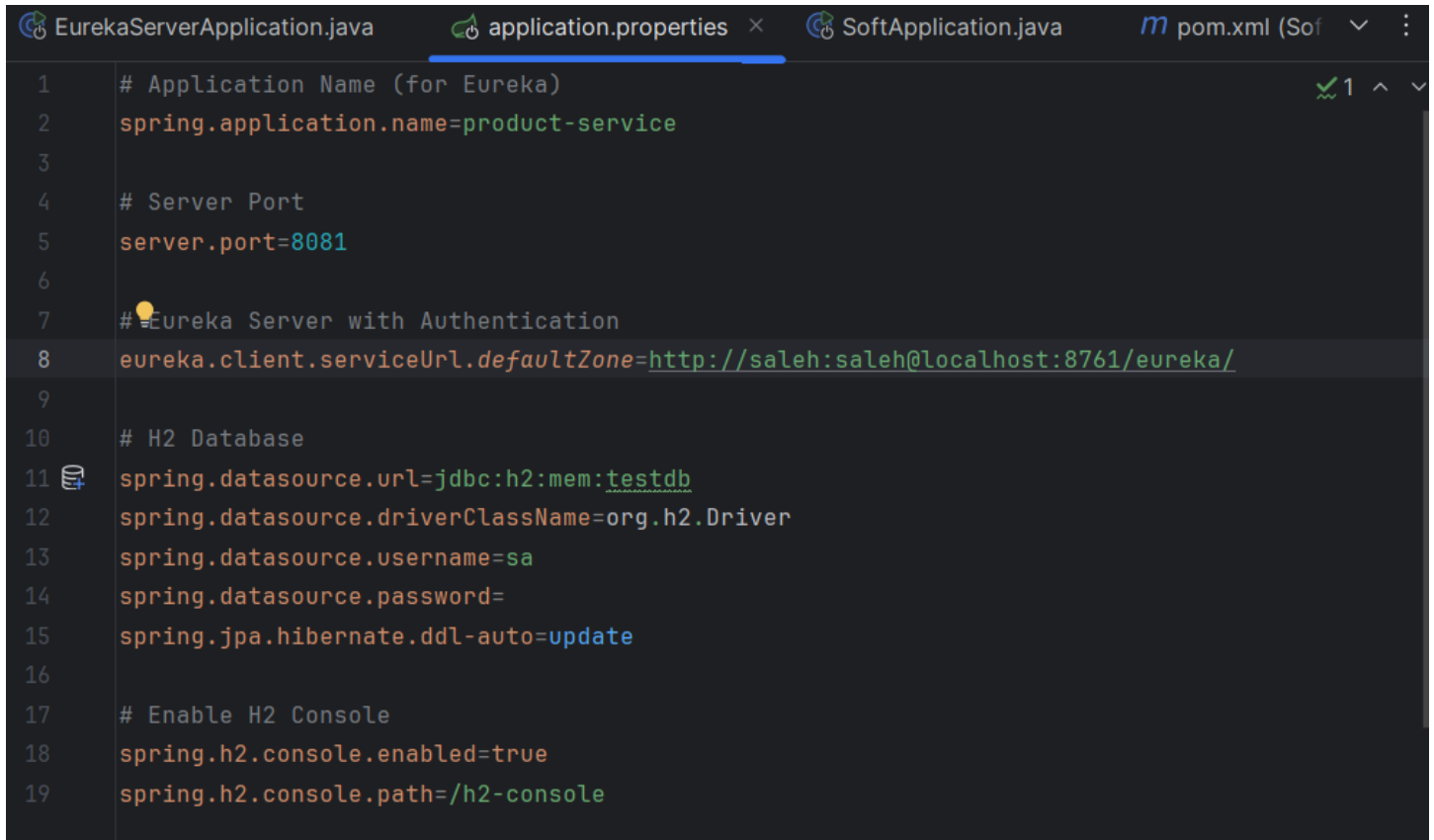
The screenshot shows the Spring Boot IDE configuration window. On the left, the 'Project' tab is active, showing settings for 'Gradle - Kotlin' and 'Maven'. The 'Spring Boot' section shows '3.4.5' as the selected version. The 'Project Metadata' section shows 'com.labfinal2' as the group, 'SCAD2' as the artifact and name, and 'Lab final for Software Construction and development.' as the description. The 'Packaging' is set to 'Jar' and the 'Java' version is '17'. On the right, the 'Dependencies' tab is active, showing a list of dependencies: 'Spring Security' (SECURITY), 'Spring Web' (WEB), 'Spring Data JPA' (SQL), 'H2 Database' (SQL), and 'Eureka Discovery Client' (SPRING CLOUD DISCOVERY). Each dependency has a red minus button to its right.

Project	Language	Spring Boot	Project Metadata	Dependencies
☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Maven	☒ Java ☐ Kotlin ☐ Groovy	☐ 3.5.0 (SNAPSHOT) ☐ 3.5.0 (RC1) ☐ 3.4.6 (SNAPSHOT) ☒ 3.4.5 ☐ 3.3.12 (SNAPSHOT) ☐ 3.3.11	Group: com.labfinal2 Artifact: SCAD2 Name: SCAD2 Description: Lab final for Software Construction and development. Package name: com.labfinal2.SCAD2 Packaging: ☒ Jar Java: ☐ 24 ☐ 21 ☒ 17	Spring Security (SECURITY) Highly customizable authentication and access-control framework for Spring applications. Spring Web (WEB) Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container. Spring Data JPA (SQL) Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate. H2 Database (SQL) Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application. Eureka Discovery Client (SPRING CLOUD DISCOVERY) A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.

2. Configuration

Software Development and Construction

- Configured application.properties:



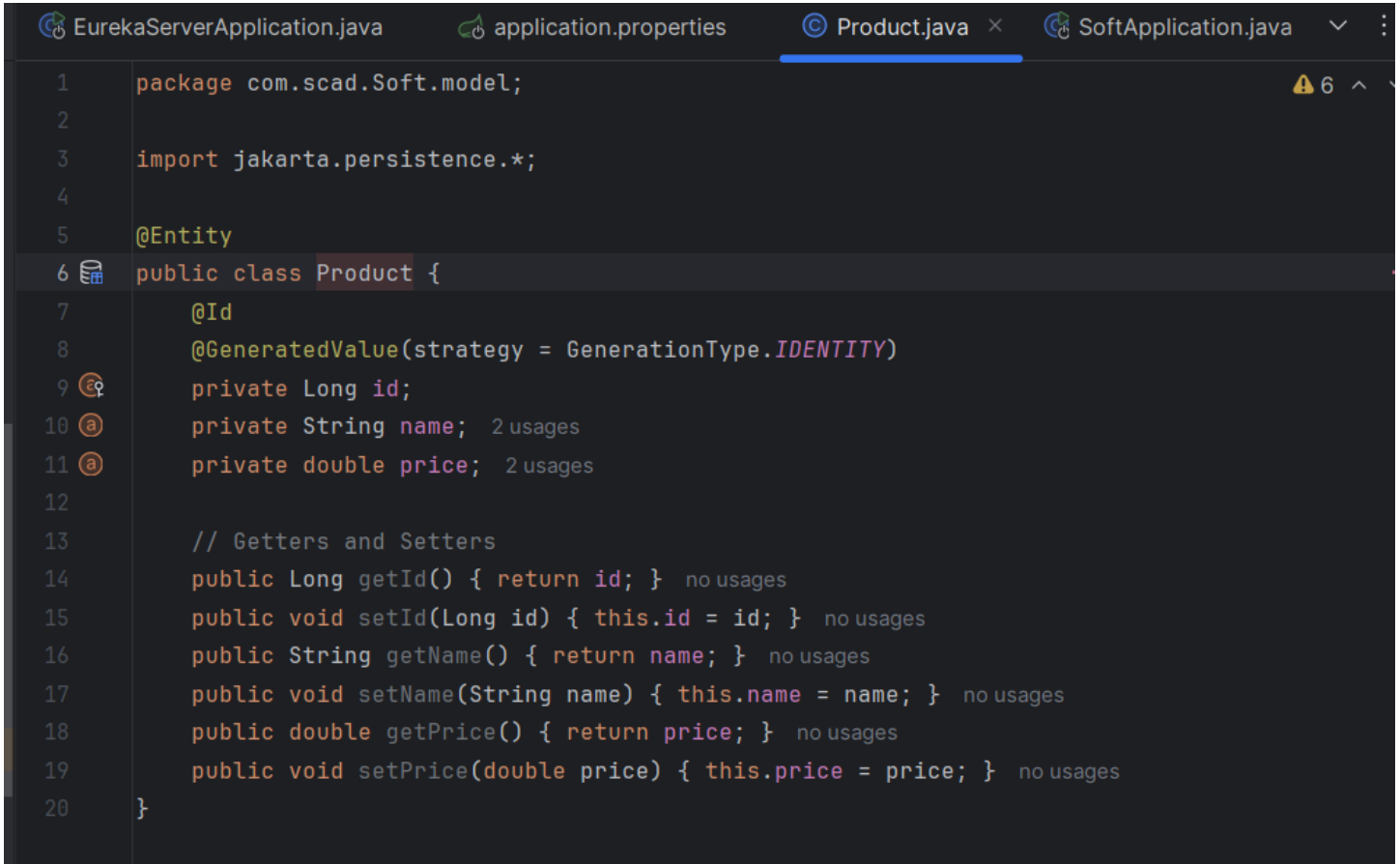
The screenshot shows an IDE with four tabs: EurekaServerApplication.java, application.properties (active), SoftApplication.java, and pom.xml (Sof). The application.properties file contains the following configuration:

```
1 # Application Name (for Eureka)
2 spring.application.name=product-service
3
4 # Server Port
5 server.port=8081
6
7 # Eureka Server with Authentication
8 eureka.client.serviceUrl.defaultZone=http://saleh:saleh@localhost:8761/eureka/
9
10 # H2 Database
11 spring.datasource.url=jdbc:h2:mem:testdb
12 spring.datasource.driverClassName=org.h2.Driver
13 spring.datasource.username=sa
14 spring.datasource.password=
15 spring.jpa.hibernate.ddl-auto=update
16
17 # Enable H2 Console
18 spring.h2.console.enabled=true
19 spring.h2.console.path=/h2-console
```

3. Code Implementation

- **Entity Class:**

Software Development and Construction



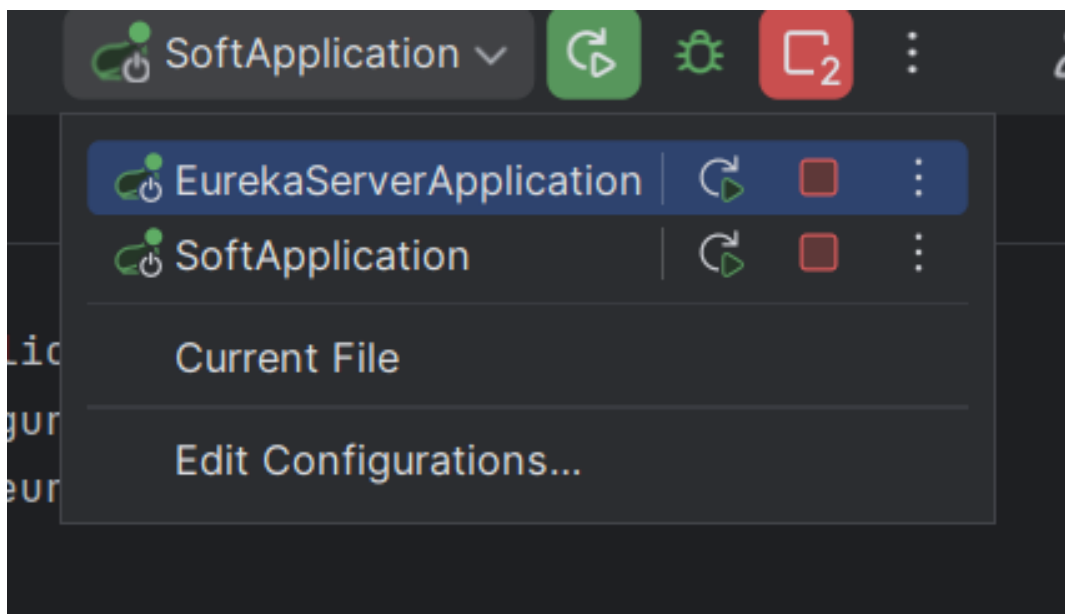
```
1 package com.scad.Soft.model;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 public class Product {
7     @Id
8     @GeneratedValue(strategy = GenerationType.IDENTITY)
9     private Long id;
10    private String name; 2 usages
11    private double price; 2 usages
12
13    // Getters and Setters
14    public Long getId() { return id; } no usages
15    public void setId(Long id) { this.id = id; } no usages
16    public String getName() { return name; } no usages
17    public void setName(String name) { this.name = name; } no usages
18    public double getPrice() { return price; } no usages
19    public void setPrice(double price) { this.price = price; } no usages
20 }
```

- **REST Controller:**

Software Development and Construction

```
1 package com.scad.Soft.controller;
2
3 > import ...
4
5 @RestController
6 @RequestMapping("/products")
7 public class ProductController {
8
9     private final ProductRepository productRepository; 3 usages
10
11     @Autowired
12     public ProductController(ProductRepository productRepository) { this.productRepository = productRepository; }
13
14     @PostMapping
15     public Product createProduct(@RequestBody Product product) { return productRepository.save(product); }
16
17     @GetMapping
18     public List<Product> getAllProducts() { return productRepository.findAll(); }
19 }
```

4. Testing



Software Development and Construction

← → ↻ ⓘ localhost:8081/h2-console/login.jsp?jsessionId=bbdc592e1ec178a04516c044c03adb72

English ▼ Preferences Tools Help

Login

Saved Settings: Generic H2 (Embedded) ▼

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:~/test

User Name: sa

Password:

Connect Test Connection

← → ↻ ⓘ localhost:8081/h2-console/login.do?jsessionId=bbdc592e1ec178a04516c044c03adb72

🔊 🛠️ ☒ Auto commit Max rows: 1000 ▼ Auto complete Off ▼ Auto select On ▼ ⓘ

jdbc:h2:mem:testdb

PRODUCT

ID

NAME

PRICE

Indexes

INFORMATION_SCHEMA

Users

H2 2.2.224 (2023-09-17)

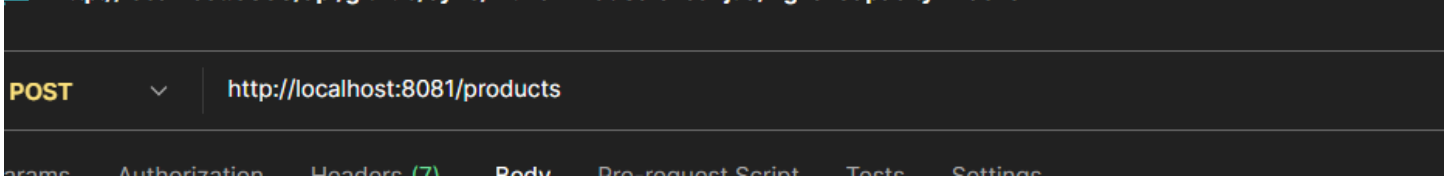
Run Run Selected Auto complete Clear SQL statement:

Important Commands

ⓘ		Displays this Help Page
		Shows the Command History
	Ctrl+Enter	Executes the current SQL statement
	Shift+Enter	Executes the SQL statement defined by the text selection
	Ctrl+Space	Auto complete
🔊		Disconnects from the database

Sample SQL Script

Software Development and Construction



Conclusion

- Successfully implemented a **secure Eureka Server** with basic authentication.
- Developed a **JPA-based microservice** that registers with Eureka and performs CRUD operations.
- Resolved version compatibility issues between Spring Boot and Spring Cloud.